仓库链接：https://github.com/zechun115/qhf（分支：大作业）

姓名：邱浩峰　　提交日期：2026 年 10 月 29 日

课 程 大 作 业 · 第 二 阶 段 （ 需 求 变 更 迭 代 ）

大作业—第二阶段项目报告

需求变更迭代 · 基于 MongoDB 的在线问卷系统

项目名称	基于 MongoDB 的在线问卷系统
技术栈	Node.js • Express • Mongoose • MongoDB
交付内容	需求变更分析 • 数据结构演进 • 可运行实现 • 自动化测试
可用性改进	多用户视角审计，修复 11 处问题（见第七节）
测试规模	92 个自动化用例全部通过（第二阶段新增 23 条）
运行方式	npm install → npm run seed → npm start（测试：npm test）

2026 年 10 月

一、需求变更概述

系统在第一阶段交付并"上线使用"后，收到用户反馈共八项需求变更。八条反馈看似零散，但指向同一个结论：**题目应当脱离问卷独立管理**。把用户原话翻译成系统语言后整理如下。

表 1-1　八项需求变更与期望效果

编号	用户反馈	期望效果
1	每次做类似问卷都要重写题目，太麻烦	可创建并保存常用题；建新问卷时可直接选用；一题可被多个问卷使用
2	团队协作只能复制粘贴	可把题目共享给其他用户；对方能在自己的问卷里使用；共享后仍可用同一题
3	改了一个题，以前问卷里的题也跟着变了，数据对不上	已发布问卷的题目不能被修改；要改就生成新题目；旧问卷保持原内容
4	改完想找回旧版本，也想知道新题从哪改来	可看历史版本；可知道来源；可恢复旧版本（v1→v2→v3 关系可见）
5	同一题想稍微改一下用在不同问卷，又不想影响原题	可基于一题创建新版本；新版本可单独修改；不同问卷可用不同版本
6	想改题但不知道哪些问卷在用它	可查看题目被哪些问卷使用；修改时自己决定是否出新版本
7	希望把常用题整理成题库	可管理题库、把题加入题库、从题库选题、题库里的题可更新版本
8	统计只能看某个问卷，想看一道题在**所有**问卷中的回答情况	单题跨问卷统计（如"你的年龄"在所有问卷中的总体分布）

按风险重新排序后，这八条其实是两类问题。需求 3、4、5 是一类，都是"改题要不要影响历史"的同一个问题的三个侧面——只要问卷侧不再依赖题库的可变状态，三条一起解决；需求 1、2、6、7、8 是另一类，都是"题目的作用域从一份问卷扩大到多份问卷"之后衍生出来的查询与权限需求。这个判断决定了后面只做两个数据结构决策，而不是八个。

任务约束：教师明确要求第二阶段必须基于第一阶段现有代码修改，不得推倒重做；报告需记录"原设计 → 新需求问题 → 为什么这样改 → 改了哪里 → 数据结构变化 → 兼容性"。因此本报告先做原设计回顾，再谈设计变更，最后才是实现与验证。

二、原设计回顾与影响分析

第一阶段有两条设计决策与本次需求直接相关，其中一条意外地成了第二阶段的地基。

其一，题目是独立集合，不内嵌在问卷里。`questions` 集合每道题一个文档，问卷只存 `question_count` 冗余计数。当时这样拆是为了编辑期并发安全——改一道题不必重写整份问卷文档。这次改动时才发现它还附赠了一个好处：题目本来就不是"某份问卷文档的附属品"，把它挂到题库上不需要动问卷文档的结构。

其二，发布后冻结题目结构。`draft → published → closed` 是单向状态机，题目增删改的所有接口入口都有 `assertDraft` 守卫。需求 3（已发布问卷题目不可修改）在第一阶段**已经由这条规则完整满足**，本次没有为它写一行新代码，只需要在新的题库场景下补一条回归验证（用例 T82）——防止后来的"从题库加题"通道绕过这道守卫。

真正的新问题是：第一阶段里题目从属于某一个问卷（`survey_id` 必填），不存在"一道题属于多个问卷"的表示方式；也没有"同一道题的多个版本"的概念。这两点必须引入新的数据结构，而引入新结构时最大的风险恰恰是需求 3——如果处理不当，题库的"更新"会反过来破坏问卷快照。所以本次设计的第一原则不是"怎么实现复用"，而是"怎么复用才不破坏历史"。

三、设计方案变更：三个核心决策

3.1 决策一："问卷侧存"快照"，不是"引用"

备选方案：问卷题只存 `bank_question_id`，读取时 join 题库的最新版本。

否决理由：这正是需求 3 要禁止的行为。题库一更新，所有已发布问卷的题目内容随之变化，历史答卷的题干、选项立即失去解释依据——"2025 年那批人选的'非常满意'到底是哪五个选项之一"这个问题再也答不上来。问题不在于代码写错，而在于"问卷题与题库题共享了同一份可变状态"。

最终方案：从题库选题时，把题目内容（题干、选项、约束）**整体拷贝**为问卷自己的题目文档；`bank_question_id` 与 `bank_version` 只作为溯源字段，记录"来自题库哪个文档的哪个版本"。此后题库无论怎么出新版本，问卷快照一个字节都不变。需求 3（旧问卷不变）与需求 5（不同问卷用不同版本）由这一个决策同时满足，且不依赖任何额外的状态同步或后台任务。

代价是存储冗余：同一道题被 100 份问卷使用就有 100 份内容拷贝。这是刻意的取舍——题目内容只有几百字节，而"历史答卷可解释"是无价的；更何况快照本来就是"当时问的是什么"的唯一权威记录，它不是缓存，不该被优化掉。

3.2 决策二：题库题"版本即文档"

修改题库题**永远不原地覆盖**，而是插入一个新文档：`version + 1`，`parent_id` 指向修改前的版本，同一棵版本树的文档共享 `root_id`（v1 的根是它自己）。

对比被否决的"数组内嵌 history"方案：内嵌数组随修改次数无限增长，逼近 16 MB 文档上限；两个并发修改会在数组层面互相覆盖（读-改-写）；更关键的是数组里的历史版本无法被"分别引用"——而需求 5 要求不同问卷分别使用 v1 和 v2，只有独立文档才有自己的 `_id` 可以被问卷溯源字段指向。

这个结构下，需求 4 的三个动作都有自然实现：看历史 = 按 `{root_id, version}` 列出版本链（`parent_id` 串出 v1→v2→v3 的关系）；知道来源 = 读 `parent_id`；恢复旧版本 = 以旧版本内容在树末端追加一个新版本（v3 内容等于 v1），历史一条不删。恢复因此是"再改一次"而不是"回滚"，这也顺带避免了"回滚后引用旧版本的问卷指向虚空"的问题。

3.3 决策三：共享是"树级"语义

`shared_with`（用户 id 数组）内嵌在题库题文档上，共享与取消共享对整棵版本树 `updateMany`。有两个容易漏掉的细节。

第一，作者共享 v1 之后又改出 v2，若 v2 不继承 `shared_with`，被共享者会"丢题"——所以新版本文档创建时显式继承来源版本的共享列表。第二，被共享者的权限是"读 + 选题进问卷 + fork"，**不能**修改原题（403）；团队协作由 `fork` 收尾：fork 生成一棵 owner 是自己的新树，之后各改各的，从根上避免写写冲突，也不需要引入锁或冲突合并。

3.4 这套方案的代价与已知边界

方案不是免费的，把边界写清楚比宣称"完美"更有用。

- **版本树只增不删**：一题改 20 次就有 20 个文档。当前以 `{root_id, version:-1}` 索引支撑"取最新版"，列表只展示最新版本以压住视觉噪音；若长期运行，需要一条归档策略（例如保留最近 N 版 + 被问卷引用过的版本）。
- **不做分支合并**：fork 出来的新树与源树此后无任何关联。这是有意的——需求要的是"各改各的"，而不是 Git 式的三方合并，后者对普通用户过重。
- **共享粒度是"整棵树"**：不能只共享 v2 不共享 v1。理由是被共享者需要看到版本演进才敢用，单版本共享反而制造"他改了我不知道"的问题。
- **快照不追溯**：题库出 v2 后，已引用 v1 的问卷不会自动升级。要升级只能由问卷作者在草稿状态重新选题——这条限制保证了需求 3，也意味着"批量升级"不在本阶段范围内。

四、数据结构变化与兼容性

4.1 新增集合 bank_questions

题库题独立成集合，而不是在 `questions` 上加一堆可空字段，原因是两者的生命周期完全不同：问卷题随问卷发布而冻结，题库题要被反复修改、共享、fork。

表 4-1　bank_questions 集合字段设计

字段	类型	说明
`owner_id`	ObjectId → users	创建者；决定修改、共享、恢复权限
`type`	String 枚举	七种题型，与 questions 一致
`title` / `required` / `options` / `constraints`	同 questions	题目内容本体
`version`	Number ≥ 1	树内版本号，从 1 开始
`root_id`	ObjectId → bank_questions	版本树根（v1 的 `_id`；v1 自身 root_id = 自身 id）
`parent_id`	ObjectId / null	修改或 fork 的来源版本；null 表示原创 v1
`shared_with`	[ObjectId]	被共享用户列表（树级语义）
`created_at` / `updated_at`	Date	时间戳

4.2 questions 集合新增字段

表 4-2　questions 集合新增的溯源字段

字段	类型	说明
`bank_question_id`	ObjectId / null	溯源：来自题库哪个文档；非题库题为 null
`bank_version`	Number / null	选题时的版本号快照

题目内容本体不变——快照在加入问卷那一刻已经完整拷贝。两个字段都只是"来源标签"，不参与任何渲染或校验计算，因此旧问卷题（两字段为 null）不需要走任何特殊代码路

径。

4.3 新增索引

表 4-3　新增索引及其支撑的查询

索引	支撑的查询
`{owner_id:1, updated_at:-1}`	我的题库列表（按更新时间倒序）
`{root_id:1, version:-1}`	版本树查询、取每棵树的最新版本
`{shared_with:1}`	"共享给我的"反查
`questions.{bank_question_id:1}`	使用情况反查、跨问卷统计的引用定位

4.4 兼容性

- **旧数据零迁移**：`bank_question_id` / `bank_version` 均可空，第一阶段创建的问卷题两个字段为 `null`，读取端按"非题库题"分支处理。第一阶段问卷、答卷、统计全部原样可用，无需跑任何脚本。
- **答卷零影响**：`responses.answers` 通过 `question_id`（问卷快照的 id）关联，与题库完全解耦；题库演进不会触碰任何答卷。
- **接口向后兼容**：`POST /surveys/:id/questions` 原有请求体（type/title/options…）行为不变；`bank_question_id` 是新增的可选字段，不传就是第一阶段的老行为。
- **失败关闭**：若 `bank_question_id` 指向不存在或无权访问的题，接口返回 400/404 而不是静默忽略——加题失败必须让用户知道，否则会得到一份少题的问卷。

五、接口设计与实现要点

新增 `routes/bank.js`，挂载于 `/api/bank-questions`，共 11 个端点覆盖八项需求。

表 5-1　题库接口与需求对应关系

需求	端点
创建 / 列表 / 详情	`POST /`、`GET /?scope=mine|shared`、`GET /:id`
修改（= 新版本）	`PUT /:id`（部分更新：未传字段继承当前版本）
多版本并存 / 接收共享	`POST /:id/fork`
版本历史 / 恢复旧版本	`GET /:id/versions`、`POST /:id/revert`
共享 / 取消共享	`POST /:id/share`、`POST /:id/unshare`
使用情况	`GET /:id/usage`
跨问卷统计	`GET /:id/stats`
问卷题存为常用题	`POST /from-question`

实现上有四个要点值得记录。

- **权限三级模型**：owner（可改、共享、恢复）、被共享者（可读、选题、fork）、其他人（403，不用 404）。这里选择 403 而不是 404 是有意的：题目 id 本身不是敏感信息，明确告知"无权访问"比伪装成"不存在"更好排障；真正敏感的问卷标题与答卷内容另有更严格的边界（见 §7.4）。所有端点入口统一走 `loadAccessibleBankQuestion()`，避免每个 handler 各写一遍判断。
- **列表去版本噪音**：题库列表按 `{root_id, version:-1}` 排序后在内存中分组取每组第一条，每棵树只展示最新版本，否则 v1/v2/v3 会把一屏铺满、用户找不到自己想要的那道题。
- **跨问卷统计是一条聚合管道**：先按溯源字段定位该题所有版本在所有问卷中的快照，然后 `$match(survey_id) → $unwind(answers) → $match(question_id)`，再按题型分支——选项类再 `$unwind option_keys` 后 `$group` 计数；数字类取 `$min/$max/$avg`；文本类 `$push` 后 `$slice` 最近 20 条；日期类按值 `$group`。计算全部下沉到数据库侧，应用层只负责拼装响应。
- **开发中修掉的两个自身设计错误**：① `root_id` 一度设为 `required`，但 v1 需要先插入拿到 `_id` 才能回填 root_id，`create()` 在插入瞬间就跑校验，直接 400——改为去掉

required、由写路径约定保证非空；② 修改接口初稿是全量语义，只传题干会把选项清空——改为部分更新，未传字段继承当前版本。

六、前端页面

题库入口加入全站导航（我的问卷 / 题目库 / 退出），像素游戏风与第一阶段完全一致：复用既有类名，零新增 CSS，视觉上不像"外挂了一个新模块"。

导航结构上有一个小决定值得说明：题库做成了与"我的问卷"平级的独立页面，而不是塞进问卷编辑页的一个侧栏。原因是两者的心智模型不同——问卷编辑是"往下写"，题库管理是"横向整理"，混在一起会让"我现在改的是这份问卷还是那道公共题"变得含糊。反过来，选题动作又必须在编辑页里完成，所以编辑页保留了一个"从题库选题"的入口，点开是一个带"我的 / 共享给我的"两个页签的弹窗。用"独立管理 + 就地选用"而不是"全部集中在一处"，是这次前端设计里唯一一次真正的结构选择。



图 6-1 题库管理页：每棵树只显示最新版本（此题为 v2）

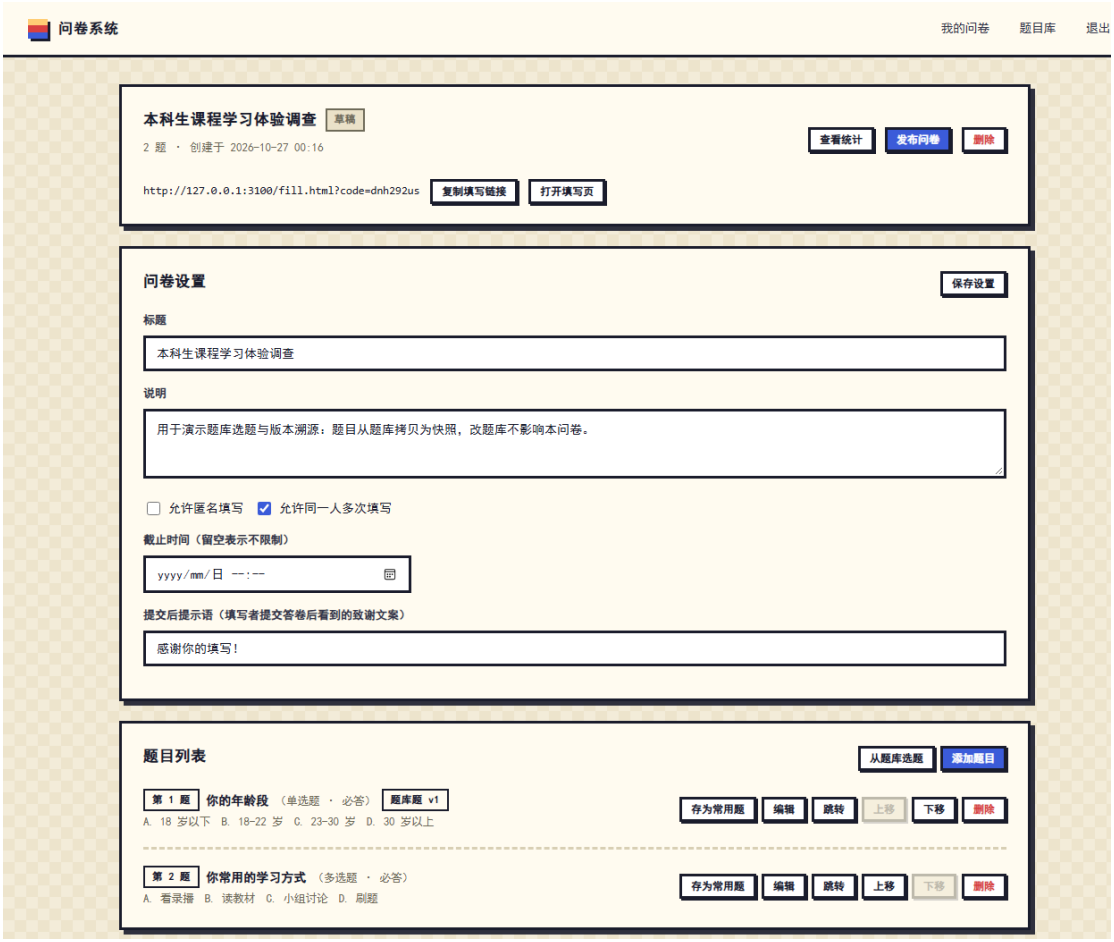


图 6-2 问卷编辑页：从题库选题与"题库题 v1"标签

页面结构上，题库页分为"我的题目 / 共享给我的"两个页签，新建题目在顶部表单完成；每道题提供六个动作，其中"版本历史"弹窗给出 v1→v2→v3 的版本链并可一键恢复旧版本，"共享"弹窗按用户名共享或取消，"使用情况"弹窗列出哪些问卷在用、用的哪个版本，"跨问卷统计"弹窗按题型展示选项分布条形图、数字 min/max/avg、文本样本与日期分布。

编辑页的改动遵循"最小侵入"：草稿状态才显示"从题库选题"按钮，弹窗内可切换"我的题目 / 共享给我的"页签并一键选用；每张题目卡片新增"存为常用题"按钮，把已写好的问卷题反向存进题库；带溯源的题只多一个"题库题 vN"小标签，其余交互与第一阶段完全相同。已发布的问卷看不到上述按钮——前端隐藏与后端 `assertDraft` 是两道独立防线，不依赖彼此。

七、多用户可用性优化

八项需求完成后，我做了一轮针对"别人来用会不会卡住"的审计。出发点很具体：这个系统不是给我一个人演示用的，第二个注册的用户、被共享题目的同事、以及手滑改坏 URL 的人，都会真实地碰到它。

7.1 审计方法

不是"读一遍代码找 bug"，而是代入三个角色各走一遍完整流程，记录每一个"这里会卡住"的瞬间。

- **第二个注册的用户**：登录后看到什么？空状态有没有引导？会不会看到别人的数据？超出一页之后还能不能翻回去？
- **被共享题目的同事**：能做什么、不该看到什么？如果他能看到出题人的问卷标题甚至答卷内容，那"共享一道题"就变成了"泄漏一份问卷"。
- **手滑的用户**：地址栏少个参数、id 被改坏、问卷已关闭还点保存，会发生什么？

7.2 发现与修复

共修复 11 处，按影响排序如下。

表 7-1　可用性审计修复清单

#	问题	影响	修复
1	列表页使用了 `toast(...)` 但解构时漏了 `toast`	复制问卷成功后抛 `ReferenceError`，列表不刷新；导出 CSV 后弹"toast is not defined"	补进解构列表
2	路径参数非 ObjectId 时 Mongoose 抛 `CastError`	直接 500"服务器内部错误"，用户看不懂也不知道找谁	全局错误处理把 `CastError` 转成 400 `BAD_REQUEST`

#	问题	影响	修复
3	题库的使用情况 / 跨问卷统计对被共享者开放	共享一次题，就能看到对方问卷标题与跨问卷汇总的答卷内容	收敛为仅出题人（403），前端对被共享题隐藏这两个按钮
4	修改题库题时顺手改了旧版本的 `updated_at`	版本历史不再只读，"历史记录"的语义被破坏	去掉对旧文档的写入，只追加新版本
5	登录页对 `next` 二次 `decodeURIComponent`	路径含 `%` 时解码错乱；且 `next=//evil.com` 可跳站外	去掉二次解码；只接受单个 `/` 开头的站内路径
6	问卷列表固定取 50 条，没有翻页	超过 50 份问卷后，更早的问卷彻底看不到	后端保留分页能力，前端补上一页 / 下一页与页码
7	题库"修改（新版本）"只能改题干	想改选项或题型只能重新建题，版本能力形同虚设	改为完整编辑弹窗（题型 / 题干 / 选项 / 必答），保存即生成新版本
8	已关闭的问卷仍显示可点的"保存设置"	点了必然 409，属于无效操作却没有任何提示	关闭后禁用截止时间与保存按钮
9	统计明细里显示受访者的数据库 ObjectId	对使用者没有意义，还暴露内部主键	改为"登录用户填写 / 匿名填写"
10	反向代理部署时 `share_url` 固定 http	https 站点复制出的填写链接协议错误，扫码打不开	`trust proxy` + `X-Forwarded-Proto`；另补三条基础安全响应头
11	已发布的问卷仍显示"添加题目""从题库选题"	用户认真填完题目表单，提交必然 409，白做一遍	发布后隐藏这两个入口；后端 `assertDraft` 仍是最终防线

这 11 条里真正会让第二个用户"用不下去"的是前三条：第 1 条是必现的功能断链，第 2 条把用户的手误变成了看起来像服务器崩溃，第 3 条则是权限边界的实际泄漏——它不是"体验不好"，而是"共享一道题却把一份问卷交出去了"。第 4 条更要紧的是语义：一个叫"历史版本"的东西如果会被后续操作改写，用户对它的信任就是错的。第 11 条则是同一种毛病的第二个例子：界面允许的动作和后端的规则不一致，用户会在点下"提交"之前一直以为这件事能做。

7.3 双账号验收

自动化测试能覆盖接口行为，但覆盖不了"两个人真的互相操作时是什么感觉"。因此另起一个独立数据库与端口，用两个真实账号把完整链路跑了一遍：注册 → 建卷 → 加题 → 存进题库 → 共享给对方 → 对方从共享题库选题 → 出题人改题 → 确认对方问卷快照未变 → 双方分别发布 → 匿名与登录各填一份 → 各自看统计。

47 项检查全部通过，其中涉及越权的 6 项均如预期返回 403（对方不能改我的题、不能看我的用量与统计、不能删我的问卷）。这一步也顺手验证了"零迁移"的说法：第一阶段的旧问卷在新代码下打开、填写、统计均正常，`bank_version` 为 null 的题目不会走进任何题库分支。

7.4 权限边界收敛

审计后明确了三条边界，写进文档也写进测试：**能看到题目**不等于**能看到使用情况**——被共享者拿到的是"一道题"，出题人手里的是"这道题连着哪些问卷、答卷里答了什么"；**能 fork** 不等于**能改原题**；**能填问卷** 不等于**能看统计**，统计只对问卷 owner 开放。这些判断都写在服务端，前端隐藏按钮只是为了不让人点出一个 403。

八、测试

新增 `tests/api.bank.test.js`，T70～T88 共 19 条用例，每条对应一项需求或一条边界；可用性优化后又补了 T89～T92 四条回归。

表 8-1　第二阶段新增用例与覆盖内容

用例组	覆盖内容
T70 – T72	创建题库题（version=1、root_id=自身）、我的列表、他人隔离与未登录 401

用例组	覆盖内容
T73 – T75	修改生成新版本（v2，parent 指向 v1）、版本链可查、revert 恢复 v1 生成 v3 且历史完整
T76 – T78	共享后对方可见、Bob 不能改 Alice 的题（403）但可 fork 后自由修改、越权 403、共享给不存在用户 404
T79 – T80	问卷题存入题库 + 从题库选题（快照 + 溯源字段）；**需求 3 核心回归：题库出 v2 后问卷快照仍为 v1 内容**
T81 – T83	使用情况反查、已发布问卷不能再从题库加题（409）、取消共享后不可见
T84 – T88	跨问卷统计完整数据流：两卷引用同一题各填答卷后，选项计数跨卷累计（2/1/1）、usage 反查、数字题 min/max/avg、未使用题返回全零
T89 – T92	可用性回归：被共享者查使用情况 403、修改题库题不改写旧版本的 `updated_at`、非法 id 返回 400 而非 500、选题传非法 `bank_question_id` 返回 400/404

全量结果：**92 条用例全部通过**（第一阶段 69 条回归无破坏 + 第二阶段 23 条新增）。用例 T80 是整套测试里最该保留的一条——它断言的不是某个接口返回什么，而是"改题库之后再读问卷，题干的每一个字都没变"。这条断言一旦失败，说明整个方案的地基塌了。

九、AI 使用记录

本阶段继续使用对话式 AI 编码助手，日志 #21～#26（完整表格见 `docs/ai-usage-log.md`）。

表 9-1　本阶段 AI 使用记录摘要

#	用途	是否修改	说明
21	八条需求的分析与方案取舍	修改后采用	AI 初稿建议"问卷引用题库并跟随最新版"，与需求 3 冲突，改为快照 + 溯源
22	题库模型与路由实现	修改后采用	修掉 root_id required 导致的 400、全量更新清空选

#	用途	是否修改	说明
			项两处错误
23	前端题库页与选题弹窗	直接采用	复用像素风类名
24	19 条测试	修改后采用	修掉二次连库挂住、漏 teardown 两处；补上 T80 关键回归
25	跨问卷统计管道	修改后采用	文本样本加 `$slice` 限量
26	多用户可用性审计与优化	修改后采用	AI 首轮只报代码风格类建议，被要求以"第二个用户登录后会发生什么"重跑，才找出 3 个真正让用户用不下去的问题

真实对话节选（完整版见 `docs/ai-usage-log.md` §5）：

- **我**：八条需求里最要紧的是"改题不能影响已做好的问卷"。题库题是不是让问卷直接引用，然后跟着最新版走？
- **AI**：可以让问卷题存 `bank_question_id` 引用题库文档，读取时 join 最新版本。这样选题复用最直接。
- **我**：不行，这正好违反第三条——老师改了题，去年发布的问卷内容就变了，收集的数据全对不上。
- **AI**：那就反过来：选题时把题目内容整体拷贝进问卷（快照），`bank_question_id` 只记来源用于反查和统计。题库题修改永远生成新版本不覆盖。这样第三条自动满足，第五条"不同问卷用不同版本"也自动成立。
- **我**：创建题库题一直 400，`details: ["root_id"]`，可我明明在 save 前回填了。
- **AI**：`Model.create()` 在插入那一刻就跑 schema 校验，此时 `root_id` 还是 null（required 校验失败），根本轮不到你后面的回填。把 `required: true` 去掉，回填改由写路径约定保证。
- **我**：我要的不是代码风格建议。假设我是第二个注册的用户，登录进来点复制问卷，会

发生什么？

- AI：（重新走查后）复制问卷的回调里调了 `toast(...)`，但页面解构时没取 `toast`，所以副本创建成功、提示抛出 `ReferenceError`，列表也不刷新——用户会以为没成功，再点一次。
- **我**：这个才是会让别人用不下去的问题。顺着"用户手滑"再走一遍：地址栏的 id 被改坏呢？
- AI：参数不是合法 ObjectId 时 Mongoose 抛 `CastError`，被全局兜底成 500"服务器内部错误"。
- **我**：换成 400 并说清"id 格式不正确"。另外检查一下共享后的可见范围——被共享的人能不能看到这道题被哪些问卷用了？
- AI：能，`usage` 和 `stats` 目前对所有可见者开放，会连带返回对方问卷的标题和汇总后的答卷内容。
- **我**：那就收回，只给 owner。共享的是题，不是问卷。

十、总结

这次需求变更是一次真正的"演进"练习：第二阶段所有改动都建立在既有代码之上——题目独立集合让题库有了挂载点，发布冻结规则让需求 3 天然满足，统一错误结构与测试基建让 23 条新用例第一天就能跑进既有套件。八项需求最终收敛为两个数据结构决策（快照溯源、版本即文档）和一个权限模型（树级共享），没有推翻任何一张已有的表，旧数据零迁移。

如果给"设计为变化留余地"找一个具体例证：第一阶段注释里写的"`schema_version` 是给第二阶段预留的"和"第二阶段要按题型扩展时只需新文档多几个字段"，在这次迭代中分别以"无需启用"和"新增 bank_questions 集合"的方式兑现——前者因为快照方案比预想的更

彻底，后者因为题库本质上是"不属于任何问卷的题目"，恰好落回第一阶段的拆分决策上。值得注意的是，这次的复用不是"预料到了所以省事"，而是"当时拆得对所以接得上"：`questions` 独立成集合的初衷只是编辑期并发安全，它在第二阶段的用处属于意外收益。

而第七节那 11 处修复提供的教训是另一类：**功能做完不等于别人能用**。八项需求全部实现、19 条测试全绿的那天，第二个用户点一下"复制问卷"就会看到一个坏掉的功能；共享一道题，会把一份问卷的统计连带交出去；界面上还摆着一个点了必然失败的发题按钮。这些问题自动化测试抓不到，因为它们不在接口层的正确性里，而在"人怎么用它"里——所以后来补的那一轮走查，价值不在于修了 11 个 bug，而在于把"换一个人来用"变成了交付前的固定动作。

附录　交付清单

表 A-1　本次提交包含的文件与位置

类别	内容	位置
后端代码	5 个数据模型、6 个路由模块、认证中间件与跳转/校验核心	`src/`
前端页面	7 个页面（登录、我的问卷、编辑、填写、统计、题库），沿用像素游戏风	`public/`
自动化测试	7 个测试文件，共 92 条用例，连接真实 MongoDB 实例	`tests/`
设计文档	README、接口文档、数据库设计、测试用例表、AI 使用日志	`docs/`
本报告	`2026_ECNU_PJ1_2_邱浩峰.pdf`，同目录附可编辑的 `.docx`	仓库根目录

本地运行三步：`npm install` → `npm run seed`（生成 demo 账号与示例数据）→ `npm start`；浏览器打开 `http://localhost:3000`，用 `demo / 123456` 登录即可看到题库与示例问卷。跑测试用 `npm test`，需本机 27017 端口有 MongoDB 在运行。